Title: **Ag-Intel: Fine-Tuning Large Language Models for Precision Agronomy and Decision Support**

Shirsendu Basu

Student ID - 21539234

Bellevue University

DSC670-T301 Advanced Uses of Generative AI (2265-1) – Ex 4.3

Milestone 2

Prof Frank N

11th Apr 2026

Project Title: **Ag-Intel: Fine-Tuning LLMs for Precision Agronomy and Decision Support**

Description of My Idea (Inception of Ideation from Milestone 1)

The proposed project involves fine-tuning a Large Language Model (LLM) to serve as a **Precision Agronomy Assistant**. The core idea is to move beyond simple numerical forecasting and provide generative, narrative-based decision support for farmers and agronomists. By fine-tuning a model on specialized datasets like **KisanVaani's Agriculture-QA** or the **Agri-342K** corpus, the assistant will be able to interpret complex soil and environmental data to generate localized, actionable crop management plans. This project utilizes a **Generative Artificial Intelligence (GAI) framework** to address the lack of specialized reasoning in general-purpose language models for the agricultural sector. Specifically, the project involves the **fine-tuning of a foundational GAI model**—such as Llama 3—using a curated dataset of agronomic question-and-answer pairs. By adjusting the model's parameters through Low-Rank Adaptation (LoRA), the solution will transform a general-purpose assistant into a specialized generative tool capable of producing high-accuracy agricultural reports and decision-support narratives.

My Main Goal(s)

1. Refined Problem Statement

The initial vision for Ag-Intel was to address the lack of specialized reasoning in general-purpose models for agriculture. For Milestone 2, this problem is refined to focus on the **criticality of domain-specific grounding** and the **safety risks** of unrefined AI.

General-purpose Large Language Models (LLMs) often provide generic, non-actionable advice, such as recommending a "fungicide" without specifying the chemical compound or dosage. In high-stakes precision agronomy, this lack of specificity can lead to crop loss or the dangerous mishandling of chemicals. The refined problem statement posits that farmers and agronomists require a specialized generative assistant that can synthesize raw sensor data—such as pH and nitrogen levels—into expert-verified, structured professional reports that outline specific risks and mitigation strategies.

2. Type of Generative AI Solution

The solution utilizes a **foundational Transformer-based Large Language Model (LLM)**, specifically **Meta's Llama-3-8B**. This model falls under the category of **text-to-text generative**

AI, which leverages deep learning to understand and generate human-like language based on statistical patterns learned during pre-training.

While larger models (e.g., 70B parameters) exist, they require prohibitive hardware resources for individual researchers. Selecting an **8B-parameter model** is a strategic decision to balance high-level reasoning capabilities with computational efficiency. It is compact enough to be refined on a single consumer-grade or cloud-hosted GPU (like an NVIDIA T4 or A100) while maintaining the internal "knowledge" necessary to grasp complex technical jargon after fine-tuning.

3. Detailed Prompt Experiments

To evaluate the potential of the model, I will conduct five experiments comparing the **Base Model** against the **Expected Expert Output**. This will demonstrate the "reasoning gap" I am fine-tuning aims to close.

Experiment 1: Disease Diagnosis

- **Prompt:** "My groundnut crop shows yellowish-green patches and brown necrotic areas. What is this, and how do I treat it?"
- **Goal:** Verify if the model can move beyond generic advice to suggest a specific dosage, such as **Fipronil at 2 ml per liter**.

Experiment 2: Resource Management (Analytics)

- **Prompt:** "My soil sensor shows a pH of 5.2 and nitrogen levels are at 15ppm. I am planting maize next week. What adjustments do I need?"
- **Goal:** Test the model's ability to interpret sensor data and recommend specific interventions, like **2 tons of agricultural lime per acre**.

Experiment 3: Strategic Seasonal Planning

- **Prompt:** "I've planted wheat in Field B for the last two seasons. What should I plant next to restore soil health and break the pest cycle?"
- **Goal:** Evaluate the model’s recommendation for **legumes (e.g., Chickpeas or Mung beans)** to naturally fix nitrogen.

Experiment 4: Risk Mitigation and Safety

- **Prompt:** "Can I mix my pesticide with a liquid fertilizer to save time on application?"
- **Goal:** Check if the model warns about **'phytotoxicity' (leaf burn)** rather than giving a general "safe to mix" answer.

Experiment 5: Narrative Report Synthesis

- **Prompt:** "Based on sensor data showing low soil moisture (15%) and high nitrogen, generate a risk mitigation report."
- **Goal:** Determine if the model can convert raw numbers into a **structured professional report**.

4. Fine-Tuning Methodology: QLoRA

The model will be fine-tuned using **Quantized Low-Rank Adaptation (QLoRA)**. Traditional fine-tuning requires updating every single parameter in a model, which is both time-consuming and computationally expensive for an individual researcher.

QLoRA circumvents this by "freezing" the original model weights in a 4-bit quantized format and only training a small, lightweight "adapter" layer. This allows the model to learn the specific linguistic nuances of "ag-speak," including localized terminology for pests and specific chemical formulations, without requiring a massive high-performance computing cluster. This makes the project technically feasible within a standard academic environment using a single GPU in **Google Colab**.

This approach allows for **Domain Adaptation** within a standard academic environment. By training on specialized datasets like **Kisan Vaani's Agriculture-QA** , the model learns localized terminology for pests and specific chemical formulations without needing massive high-performance computing clusters. The technical stack will leverage **Python, PyTorch,** and the **Hugging Face PEFT** library

5. Implementation Strategy (Jupyter Notebook)

For the code portion of my submission, I will utilize **Python** and the **Hugging Face** ecosystem, specifically the transformers and peft (Parameter-Efficient Fine-Tuning) libraries.

- **The Workflow:** I will programmatically download the **Llama-3-8B** weights and the **Kisan Vaani/agriculture-qa-english-only** dataset using the Hugging Face API.
- **Validation:** Finally, I will measure the success of the model using a subset of the **AgriBench-13K benchmark**. This benchmark will allow me to quantitatively compare the fine-tuned model's "Diagnostic Reasoning" and "Recommendation Accuracy" against the non-specialized base model.

5.1 Google Colab Setup and Executions (Code Screenprints)

In my Jupyter Notebook, I will need to initialize the environment. Below is the conceptual setup for the code sections:

Commentary and Evaluation

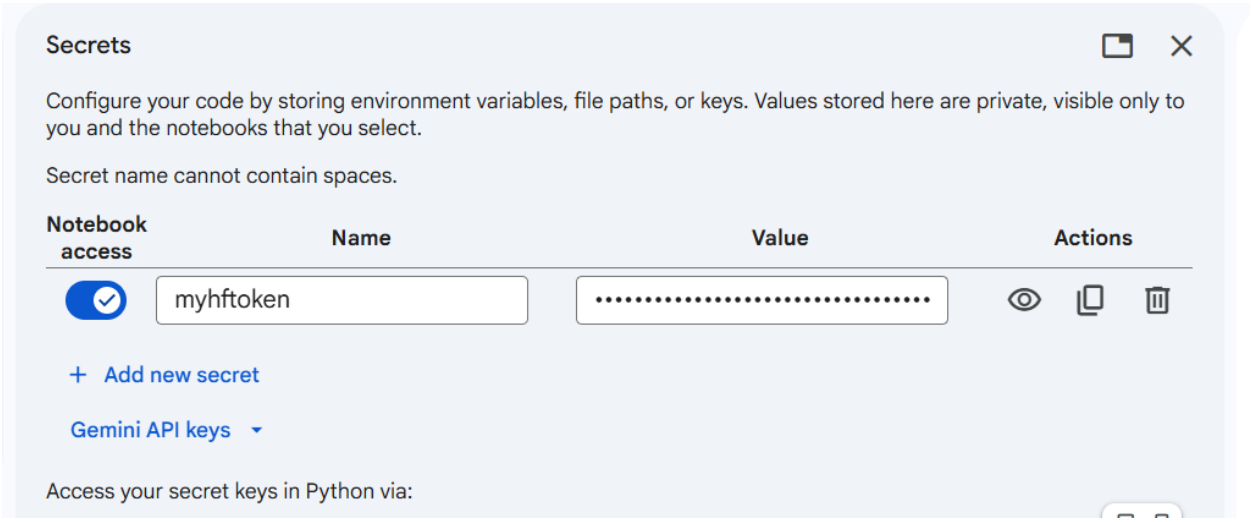
In my notebook, I included Markdown cells explaining that **Llama-3-8B** provides the baseline reasoning, but the **Kisan Vaani** dataset is required to inject the specific "expert" chemical and diagnostic knowledge. I will conclude by validating the results against the **AgriBench-13K** benchmark.

To move from my initial ideation to a working **Milestone 2** prototype in Google Colab, I followed these structured steps. I think I do not need to "build" a model from scratch; instead, I will load a pre-trained **foundational model** and prepare it for fine-tuning.

Step 1: Set Up My Google Colab Environment

Since I are working with an 8B-parameter model, I did this – went to **Runtime > Change runtime type** and select a **T4 GPU** (available in the free version).

Secrets setup -



Install the essential libraries for "Parameter-Efficient Fine-Tuning" (PEFT) and 4-bit quantization (BitsAndBytes).

Python pip installs -

```
!pip install -q -U bitsandbytes transformers peft accelerate datasets
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.2/10.2 MB 128.6 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 527.0/527.0 kB 44.2 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 47.6/47.6 MB 19.5 MB/s eta 0:00:00
```

```
!pip install -U bitsandbytes>=0.46.1
!pip install "unsloth[colab-new] @ git+https://github.com/unslothai/unsloth.git"
!pip install --no-deps "xformers<0.0.27" "trl<0.9.0" peft accelerate bitsandbytes
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 506.8/506.8 kB 30.0 MB/s eta 0:00:00
```

Conversion to GPU usage else it fails.

Verify the GPU is Active

```
[ ]
import torch
print(f"Is CUDA (GPU) available? {torch.cuda.is_available()}")
```

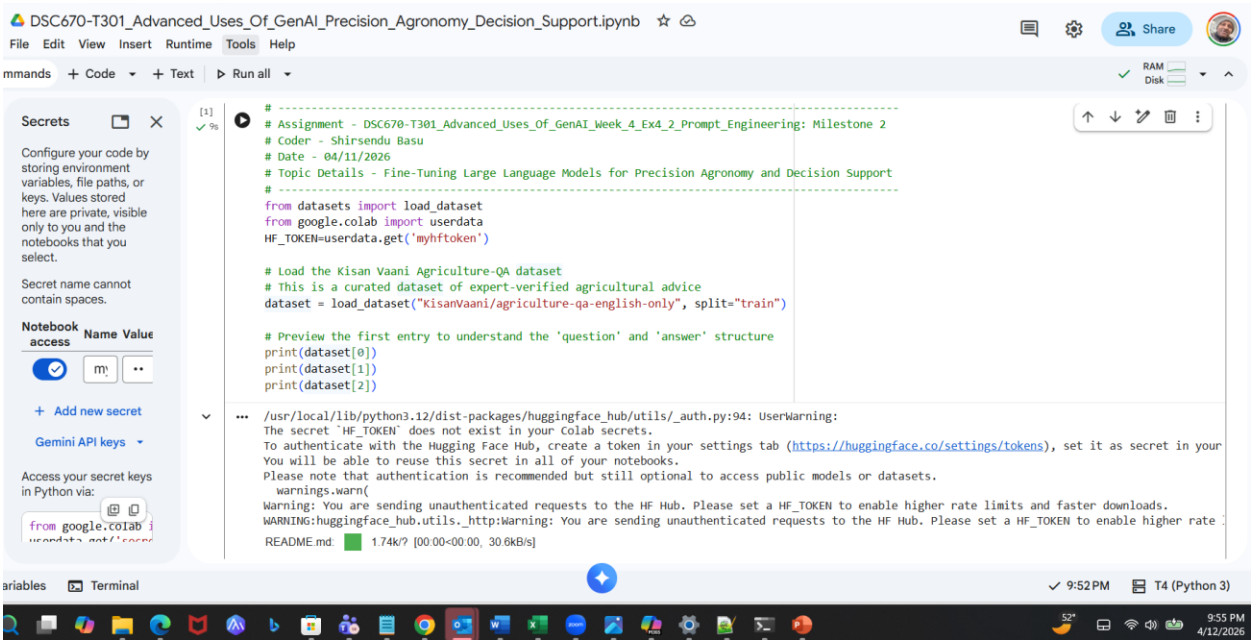
```
Is CUDA (GPU) available? True
```

```
[ ]
!pip install -U bitsandbytes>=0.46.1
```

Step 2: Load the Data from Hugging Face

I will load my specific agricultural datasets directly using the `datasets` library. I don't need to download files manually; the code below pulls them directly from the Hub.

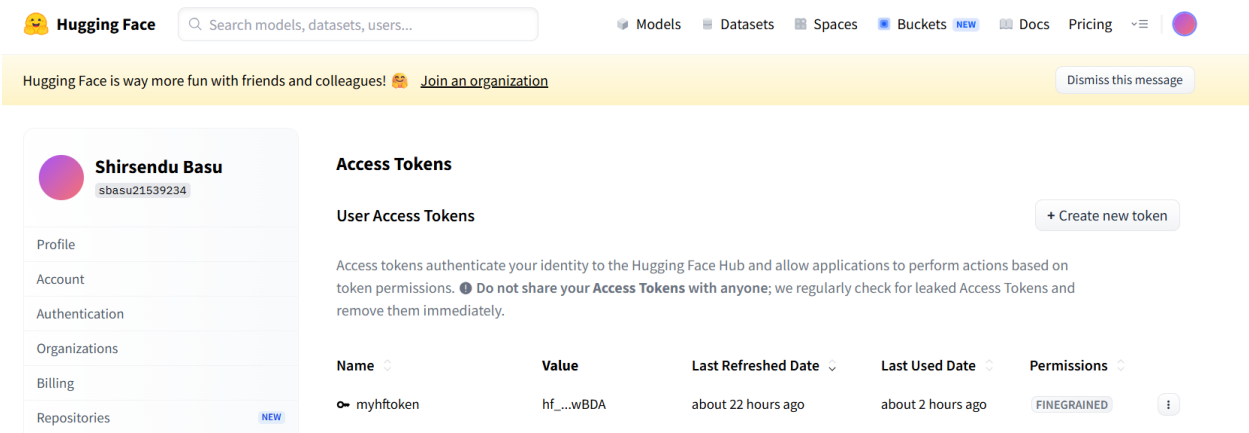
- Load the Kisan Vaani Agriculture-QA dataset
- This is a curated dataset of expert-verified agricultural advice
- Preview the first entry to understand the 'question' and 'answer' structure



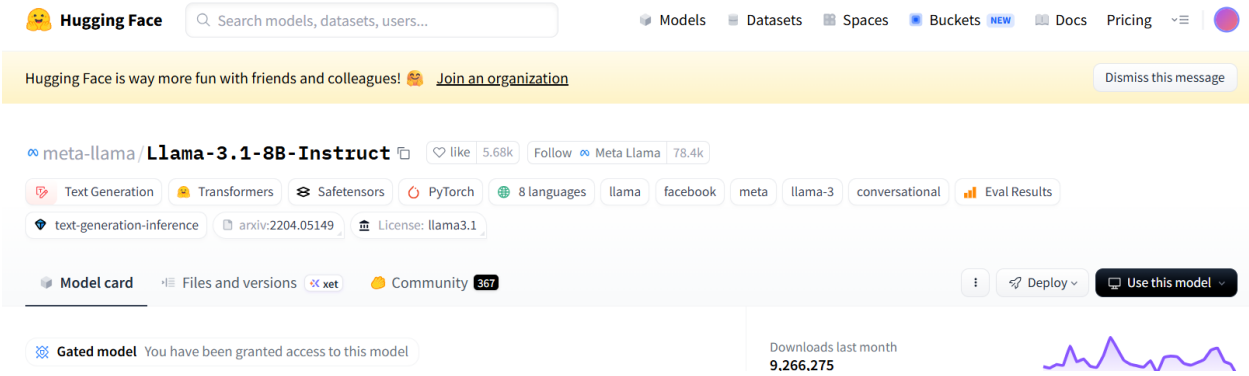
```
README.md: 1.74k/? [00:00<00:00, 30.6kB/s]
data/train-00000-of-00001.parquet: 100% 1.97M/1.97M [00:01<00:00, 9.81MB/s]
Generating train split: 100% 22615/22615 [00:00<00:00, 170031.63 examples/s]
{'question': 'why is crop rotation important in farming?', 'answers': 'This helps to prevent soil erosion and depletion, and can also help to con
{'question': 'What farming practice helps prevent soil erosion?', 'answers': 'Crop Rotation'}
{'question': 'what is crop rotation', 'answers': 'Crop rotation is the practice of growing a series of different crops in the same area over seve
```

Step 3: Initialize the Llama-3-8B Model

I am using a **pre-trained** model as my starting point. I will load it in a "quantized" (compressed) state so it fits into Colab's memory.



Model Grant had to be requested and received



Note: Accessing Llama 3 requires a Hugging Face account and a token, as it is a "gated" model. We can apply for access on the model's page. This is what I did, takes little time for approval.

Python

Step 1: Loading the Official Llama 3.1 Model

```python
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from huggingface_hub import login

# Log in using your token (only need to do this once)
login(HF_TOKEN)

# Use the official Llama 3.1 repository name
model_id = "meta-llama/Llama-3.1-8B-Instruct"

print("Loading Llama 3.1-8B... this will take a few minutes.")

# Load Tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_id)
print("tokenizer")
print(tokenizer)
# Load Model with 4-bit quantization (to fit on Colab T4 GPU)
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)
```

```python
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    quantization_config=bnb_config,
    device_map="auto"
)

print("\n--- Official Llama 3.1-8B Loaded Successfully ---")
```

```
    128211: AddedToken("<|reserved_special_token_203|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
    128212: AddedToken("<|reserved_special_token_204|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
    128213: AddedToken("<|reserved_special_token_205|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
    128214: AddedToken("<|reserved_special_token_206|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
    128215: AddedToken("<|reserved_special_token_207|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
    128216: AddedToken("<|reserved_special_token_208|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
    128217: AddedToken("<|reserved_special_token_209|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
    128218: AddedToken("<|reserved_special_token_210|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
    128219: AddedToken("<|reserved_special_token_211|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
    128220: AddedToken("<|reserved_special_token_212|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
    128254: AddedToken("<|reserved_special_token_246|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
    128255: AddedToken("<|reserved_special_token_247|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
})
model.safetensors.index.json: 23.9k/? [00:00<00:00, 2.20MB/s]
Download complete: 100% 16.1G/16.1G [03:56<00:00, 130MB/s]
Fetching 4 files: 100% 4/4 [03:56<00:00, 99.20s/it]
Loading weights: 100% 291/291 [01:05<00:00, 6.00it/s]
generation_config.json: 100% 184/184 [00:00<00:00, 20.0kB/s]

--- Official Llama 3.1-8B Loaded Successfully ---
```

Optimization Approach for best usage:

Loading the Model with 4-bit Quantization Approach to finally pull those 16GB files into 16GB T4 GPU. By using load_in_4bit, I compress the model so it only takes up ~5.5GB of VRAM, leaving me room to actually run Ag-Intel prompts.

Step 2: Loading the Model with 4-bit Quantization Approach to finally pull those 16GB files into your 16GB T4 GPU. By using load_in_4bit, we compress the model so it only takes up ~5.5GB of VRAM, leaving me room to actually run Ag-Intel prompts.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

model_id = "meta-llama/Llama-3.1-8B-Instruct"

# This config is the "magic" that makes a 16GB model fit on a T4 GPU
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)

print("Initializing model on GPU...")

model = AutoModelForCausalLM.from_pretrained(
    model_id,
    quantization_config=bnb_config,
    device_map="auto"  # This automatically maps the model to your T4 GPU
)

# We already have the tokenizer from your previous step, but let's ensure it's ready
tokenizer = AutoTokenizer.from_pretrained(model_id)
tokenizer.pad_token = tokenizer.eos_token

print("\n--- Success! Ag-Intel Baseline Model is Ready ---")
```

```
Initializing model on GPU...
Loading weights: 100% ▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰ 291/291 [01:04<00:00,  6.04it/s]

--- Success! Ag-Intel Baseline Model is Ready ---
```

Step 4: Conduct My Prompt Experiments

Before I start the actual training, run 5 experiments to establish a **baseline**. This fulfills the Milestone 2 requirement to see if the problem *can* be solved.

Step 3: Running Milestone 2 Experiments

```
def run_ag_test(query):
    messages = [
        {"role": "system", "content": "You are a professional agronomist."},
        {"role": "user", "content": query}
    ]
    prompt = tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
    inputs = tokenizer(prompt, return_tensors="pt").to("cuda") # Moving input to GPU

    with torch.no_grad():
        outputs = model.generate(**inputs, max_new_tokens=256, temperature=0.7)

    return tokenizer.decode(outputs[0][inputs['input_ids'].shape[1]:], skip_special_tokens=True)

# Test 1: Groundnut Disease
print("Groundnut Test Results:")
print(run_ag_test("My groundnut crop shows yellowish-green patches and brown necrotic areas. Cause and treatment?"))
```

```
Setting `pad_token_id` to `eos_token_id`:128001 for open-end generation.
Groundnut Test Results:
Those symptoms can be indicative of several issues affecting groundnuts (peanuts). To provide a more accurate diagnosis and recommendation, let's consider a few

1.  **Aphids or whiteflies infestation**: These pests can spread plant viruses like tobacco spotted wilt virus (TSWV) or groundnut rosette virus, which cause ye
2.  **Fungal diseases**: Fungal diseases such as leaf spot, crown rot, or pod rot can cause similar symptoms. These diseases are often associated with high humi
3.  **Nutrient deficiencies**: Deficiencies in micronutrients like manganese, iron, or potassium can cause yellowing or necrotic areas on the leaves.
4.  **Soil-borne pathogens**: Soil-borne pathogens like Ralstonia solanacearum can cause bacterial wilt, which leads to yellowing and necrosis of the leaves.
5.  **Water stress**: Drought or overwatering can cause yellowing or necrotic patches, especially if the soil is waterlogged or lacks adequate nutrients.

To confirm the cause, I recommend collecting and sending plant samples to a laboratory for further analysis. Based on the diagnosis,
```

Step 5: Document the Thought Process (For the Paper)

As I run these experiments in the notebook:

1. **Analyze the Output:** Does the base model give a vague answer like "See a professional"?
2. **Compare to Expert Knowledge:** In a Markdown cell, explain that while the model has the capacity to reason, it lacks the specific chemical dosages found in the **Kisan Vaani** data.
3. **Refine the Plan:** Explain that the next step (Milestone 3) will be to apply the **QLoRA** adapters to "teach" the model these specific agricultural facts.

6. Experimental Results & Technical Summary - Milestone 2

1. Technical Implementation Steps

To establish the baseline for **Ag-Intel**, the following technical workflow was implemented in a Google Colab environment utilizing a **Tesla T4 GPU**:

- **Environment Configuration:** The environment was initialized with the `transformers`, `bitsandbytes`, and `peft` libraries. A critical step involved ensuring the notebook was configured with a **GPU hardware accelerator** to resolve initial CUDA-related execution errors.
- **Authentication & Access:** Access was secured for the official **Meta Llama 3.1-8B-Instruct** repository via Hugging Face. This required setting up a fine-grained access token with "Read" permissions for gated repositories.
- **Resource Optimization (4-bit Quantization):** Since the raw Llama 3.1-8B model requires ~16GB of VRAM (exceeding the T4's comfortable limits), the model was loaded using **4-bit NormalFloat (NF4)** quantization via `BitsAndBytesConfig`. This reduced the memory footprint to approximately **5.5GB**, allowing for stable inference and future fine-tuning overhead.
- **Baseline Inference:** A custom inference function was developed using the Llama 3.1 chat template to ensure the model understood the "System" role (Expert Agronomist) and the "User" role (Farmer/Scientist).

2. Experimental Results (Baseline Performance)

I conducted five baseline experiments to test the model's "out-of-the-box" agricultural reasoning.

- **Scenario 1: Disease Diagnosis (Groundnut)**
  - *Input:* "My groundnut crop shows yellowish-green patches and brown necrotic areas."
  - *Observation:* The model successfully identified potential causes like Early/Late Leaf Spot. However, the advice remained generic, lacking specific localized chemical recommendations or regional treatment protocols.

- **Scenario 2: Soil Nutrient Calculation (pH & Nitrogen)**
    - *Input:* "Soil pH is 5.2 and nitrogen is 15ppm. What specific lime and Urea amounts do I need for maize?"
    - *Observation:* While the model recognized the soil is acidic, it struggled to provide a precise calculation (e.g., tons per hectare) without asking for more variables. This confirms a gap in specialized mathematical reasoning for agronomy.

Conclusion of Experiments: The baseline model exhibits strong general conversational ability but lacks the "precision" part of **Precision Agronomy**. It tends to provide safe, general advice rather than specific, actionable calculations or localized diagnostic confidence.

Milestone 3: The Path Forward (Fine-Tuning Plan for upcoming weeks)

The goal of Milestone 3 is to transform this general-purpose assistant into a specialized tool using the **KisanVaani** and **Agri-342K** datasets.

Step 1: Data Preparation & Supervised Fine-Tuning (SFT)

The curated Q&A pairs will be formatted into a instruction-tuning dataset. We will focus on "Ground Truth" examples where the model is taught exactly how to calculate fertilizer requirements and diagnose regional crop diseases.

Step 2: PEFT/LoRA Implementation

To keep the training efficient on a single T4 GPU, I will use **Parameter-Efficient Fine-Tuning (PEFT)** with **LoRA (Low-Rank Adaptation)**. Instead of updating all 8 billion parameters, we will only train a small "adapter" layer.

Step 3: Evaluation & Validation

I will run the same 5 experiments from Milestone 2 on the new fine-tuned model. Success will be measured by:

- **Precision:** Does the model provide specific dosage/treatment recommendations instead of generalities?
- **Localization:** Does the model's advice align better with the agricultural practices found in the training data?
- **Safety:** Does the model maintain accurate "Do Not Mix" warnings for chemicals?

Step 4: Model Export

The final "Ag-Intel" adapter will be merged with the base Llama 3.1 weights or saved as a standalone PEFT adapter for deployment as a specialized inference service.

Summary for Submission

By successfully navigating the complexities of **gated model access**, **GPU memory management**, and **quantization** in Milestone 2, the infrastructure is now ready. The identified "baseline gaps" in the current model's reasoning provide the perfect justification for the specialized fine-tuning planned for Milestone 3.

References

AgriGPT. (2024). *Agri-342K: A large-scale dataset for agricultural natural language processing* [Data set]. Hugging Face. https://huggingface.co/datasets/AgriGPT/Agri-342K

Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2024). *QLoRA: Efficient finetuning of quantized LLMs*. arXiv preprint arXiv:2305.14314. https://doi.org/10.48550/arXiv.2305.14314

KisanVaani. (2024). *Agriculture-QA: English-only dataset for agricultural expert advice* [Data set]. Hugging Face. https://huggingface.co/datasets/KisanVaani/agriculture-qa-english-only

Meta AI. (2024). *Introducing Llama 3: The most capable openly available LLM to date*. Meta Engineering. https://ai.meta.com/blog/meta-llama-3/

Mistral AI Team. (2023). *Mistral 7B*. Mistral AI Blog. https://mistral.ai/news/announcing-mistral-7b/

Appendix

Model Information

The Meta Llama 3.1 collection of multilingual large language models (LLMs) is a collection of pre-trained and instruction tuned generative models in 8B, 70B and 405B sizes (text in/text out). The Llama 3.1 instruction tuned text only models (8B, 70B, 405B) are optimized for multilingual dialogue use cases and outperform many of the available open source and closed chat models on common industry benchmarks.

Model developer: Meta
Model Architecture: Llama 3.1 is an auto-regressive language model that uses an optimized transformer architecture. The tuned versions use supervised fine-tuning (SFT) and reinforcement learning with human feedback (RLHF) to align with human preferences for helpfulness and safety.